Bazar.com

Distributed and Operating Systems · Fall 2026
Lab 1 — Design, implementation and execution

1. Architecture and responsibilities

The system implements the four-book store as three independent Python Flask microservices. Waitress provides a threaded HTTP server. Every communication between components uses HTTP REST and JSON; neither frontend nor order imports catalog state. Docker Compose starts three Linux containers on one private network, with only frontend port 5000 published to the host.

```
Host client → Frontend :5000
Frontend → Catalog :5001 (search / info)
Frontend → Order :5002 (purchase)
Order → Catalog :5001 (query / update)
Catalog → catalog.db · Order → orders.db
```

The frontend routes search and information requests to catalog and purchase requests to order. Catalog owns book IDs, exact titles, topic, quantity and price. Books 1–2 belong to distributed systems; books 3–4 belong to undergraduate school. The initial stock is ten per title. Seed prices are 40, 50, 35 and 25 currency units; these values are implementation choices because the assignment does not specify prices. Inserts use INSERT OR IGNORE so service restarts cannot reset existing inventory.

2. Interfaces and request flow

Service	REST operation	Behavior
Frontend	GET /search/<topic> GET /info/<id> POST /purchase/<id>	JSON responses forwarded from the appropriate backend
Catalog	GET /query/topic/<topic> GET /query/item/<id>	Aliases /search and /info match the appendix
Catalog	PATCH /update/<id>	stock_delta and/or price in JSON
Order	POST /purchase/<id>	Query availability, decrement, record receipt

A search returns an array of ID/title objects. Info returns ID, title, topic, quantity and price. Successful purchases return HTTP 201, order ID, book title and the required “bought book book_name” message. Unknown IDs produce 404, insufficient stock 409, invalid update data 400 and unavailable upstreams 503. Unknown topics return an empty array. All HTTP errors produced by Flask are encoded as JSON.

3. Persistence, concurrency and tradeoffs

Each backend uses its own lightweight SQLite database. Catalog records price as integer cents, avoiding cumulative binary floating-point errors during storage. The REST representation uses ordinary JSON numbers. Orders contain a generated primary key, book ID, title snapshot and creation timestamp. Named Docker volumes keep both databases across container replacement. No heavyweight database or GUI is required.

Waitress accepts concurrent requests using worker threads. A request obtains its own SQLite connection, with a 15-second lock timeout. Catalog updates begin an immediate transaction before reading the current quantity, validating the proposed change and writing it. This serializes competing stock changes. Crucially, order's preliminary query is advisory: catalog validates availability again inside the update transaction. Two buyers therefore cannot both decrement the last copy. Failed transactions roll back automatically.

Order first performs the required catalog lookup. If stock is positive, it asks catalog to subtract one. Only a successful decrement is followed by an order-log insertion. This meets normal-operation behavior while preserving stock safety under concurrency. The integration scenario sends 30 requests for ten copies and checks for exactly ten receipts, twenty conflicts and zero remaining stock.

4. Known limitations and failure cases

The catalog update and order insert are separate local transactions. A crash after the catalog commit but before the order insert can lose one unit without a receipt. The code logs insertion failures as reconciliation incidents; it does not claim an atomic distributed purchase. A network timeout can also leave the outcome ambiguous after a server committed. Purchases have no idempotency key, so blindly retrying can buy twice. These are explicit limitations of this small lab implementation.

There is no authentication, TLS termination, payment processing, cancellation or shipping workflow. Stock-update endpoints are intended for trusted backend operators and are not published on a host port by Compose. Health probes confirm process availability rather than full dependency readiness. Topic matching is case-insensitive exact matching. IDs are positive integer path values. Load beyond SQLite's practical write capacity can exhaust lock timeouts; this submission is sized for a small store, not large-scale production.

5. Improvements and extensions

A robust next version would accept a client-generated purchase key and persist catalog reservations under that key. Retrying a reservation would return the previous result, then order could record a unique matching receipt. A durable reconciliation worker could complete pending orders or release stale reservations. This is preferable to an unsafe automatic compensating increment after an ambiguous timeout.

Additional improvements include authentication for inventory management, structured request IDs propagated between services, metrics for latency and error rates, and a durable restock feed that applies delivery IDs only once. Multiple frontend instances can be added behind a load balancer. Database scaling would require a different persistence strategy; sharing a SQLite file across machines is intentionally avoided.

6. Running the program

Install Docker with Linux-container support. Open a terminal in the project directory and run `docker compose up --build -d`. Compose builds the Python image, creates two persistent volumes, starts catalog, waits for catalog health, starts order and then frontend. Check status with `docker compose ps`. The host reaches the REST API at `http://localhost:5000`.

For the provided host client, create a Python virtual environment, activate it and run `python -m pip install -r requirements.txt`, followed by `python client.py`. It prints both topic searches, book information, a purchase message and the updated information. Alternatively use curl or a browser for GET requests. Purchase requires POST, for example `curl -X POST http://localhost:5000/purchase/2`. On Windows PowerShell use `curl.exe`.

The README provides the inventory update command and a local three-process alternative. Services can run on separate hosts by changing CATALOG_URL and ORDER_URL and binding each HTTP server to its reachable interface. Frontend and order require only network URLs; they never access another service's database.

7. Evaluation and reproducibility

Run `python verify.py` to start three independent Waitress processes on ports 15500–15502 with isolated temporary databases. It checks both searches, empty results, info, successful purchase, unknown IDs, restock, price change, invalid input, stock exhaustion, 30 concurrent purchases, database persistence after process restarts and unavailable upstream behavior. The test deliberately isolates itself from normal application data and terminates all its child processes.

Actual captured integration output is in `docs/output.txt`. Service HTTP messages and “bought book” messages are in `docs/service-output.txt`. Docker verification and host-client evidence are recorded separately in the docs directory. These files represent real execution; expected responses have not been substituted for measured output. The exact verification status is in `docs/verification.md`.

Use `docker compose logs --no-color` to inspect component logs. Use `docker compose stop` to stop the application without removing data, and `docker compose up -d` to resume. Avoid `docker compose down -v` unless intentionally resetting all catalog and order data.

8. Submission and source control

The package contains commented source, Dockerfile, Compose configuration, a host client, integration verification, this report and actual logs. Add your name and student identifier before submitting. The repository is https://github.com/NOOROWAISSI/bazar. The core implementation was imported from an existing local lab package. Subsequent commits document actual validation fixes, tests and documentation. This does not retroactively demonstrate GitHub use throughout the original development.

> The three-service functionality and data persistence are implemented. Known cross-service crash windows are documented so the system's guarantees remain clear and testable.